Basic (your first LCEL chain) — Solution

> Back to the exercise · Hint 1 · Hint 2 · Solution

Story — `lcel_chain_basics_practice.py`: this 3-piece chain — prompt, model, parser, joined with `|` — is the atom every LangChain thing you'll ever build is made of. Written here, once, on its own, so the `|` syntax and what each piece does is obvious before Doc06's tools, Doc08's RAG pipeline, or this document's own Build Task pile more onto it. **If not:** the first time you'd see `prompt | model | parser` would be inside a bigger chain already doing three other things, making it hard to tell which part the `|` syntax actually changed.

Basic Version

Approach 1 — the direct way

```
# lcel_chain_basics_practice.py
from langchain_core.prompts import ChatPromptTemplate
from langchain_openai import ChatOpenAI
from langchain_core.output_parsers import StrOutputParser

prompt = ChatPromptTemplate.from_template("Answer: {question}")
model = ChatOpenAI()
parser = StrOutputParser()

chain = prompt | model | parser

result = chain.invoke({"question": "What's 5 + 7?"})
print(result)
```

Expected output:

```
5 + 7 = 12
```

This version works correctly. It's missing an explicit model name and temperature (it relies on `ChatOpenAI()`'s hidden defaults), and it builds the pieces as separate named variables before chaining — fine for a first working version, but not quite what you'd write once the pattern feels natural.

> Back to the exercise · Hint 1 · Hint 2 · Solution

Intermediate Version

Approach 1 — a small `build_chain()` function, with explicit settings

```
# lcel_chain_basics_practice.py
from langchain_core.prompts import ChatPromptTemplate
from langchain_openai import ChatOpenAI
from langchain_core.output_parsers import StrOutputParser


def build_chain(model_name: str = "gpt-4o-mini", temperature: float = 0.0):
    # why: model name and temperature as real parameters, not hidden
    # defaults — a model swap should be a visible, one-line decision.
    prompt = ChatPromptTemplate.from_template("Answer: {question}")
    model = ChatOpenAI(model=model_name, temperature=temperature)
    parser = StrOutputParser()
    # how: | connects the three pieces into one Runnable — the prompt's
    # output feeds the model, the model's output feeds the parser.
    return prompt | model | parser


if __name__ == "__main__":
    chain = build_chain()
    result = chain.invoke({"question": "What's 5 + 7?"})
    print(f"Answer: {result}")
```

Expected output:

```
Answer: 5 + 7 = 12
```

Difference from Basic: the model name and temperature are explicit parameters instead of hidden defaults — this matters because a model swap or a temperature change should be a visible, one-line decision, not something buried in a library default that might change between versions. Wrapping the chain-building in a function (`build_chain()`) also means this exact chain can be imported and reused elsewhere, which is exactly what this document's Build Task asks you to do.

> Back to the exercise · Hint 1 · Hint 2 · Solution

Approach 2 — the key from Doc01's `Config`, passed in or loaded

Story — `build_chain(config: Config | None = None)`: the key should come from the one place that checked it — Doc01's `load_config()`. Making `config` an optional parameter whose default is `None` lets normal code call `build_chain()` with nothing, while a test passes its own `Config` and never needs a `.env` file. **If not:** every test of the chain would need a real key in `.env` just to build it.

Copy Doc01's `config.py` and `exceptions.py` into `practice/`, unchanged, and run from inside `practice/`.

```
# lcel_chain_basics_practice.py
from langchain_core.output_parsers import StrOutputParser
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.runnables import Runnable
from langchain_openai import ChatOpenAI

from config import Config, load_config


def build_chain(config: Config | None = None) -> Runnable:
    # why: None as the default means "load the real settings from .env";
    # a test can pass its own Config instead, without touching .env
    if config is None:
        config = load_config()
    prompt = ChatPromptTemplate.from_template("Answer: {question}")
    # how: the key comes from Doc01's checked Config, not from os.environ
    model = ChatOpenAI(
        model="gpt-4o-mini", temperature=0, api_key=config.openai_api_key
    )
    return prompt | model | StrOutputParser()


if __name__ == "__main__":
    chain = build_chain()
    result = chain.invoke({"question": "What's 5 + 7?"})
    print(f"Answer: {result}")

    # how: a made-up Config — the chain is built with no .env at all
    test_config = Config(openai_api_key="sk-test-not-real", log_level="INFO")
```

```
    test_chain = build_chain(test_config)
    print(f"Built with a test Config: {type(test_chain).__name__}")
```

Expected output (the answer varies):

```
Answer: 5 + 7 = 12
Built with a test Config: RunnableSequence
```

Difference from Approach 1: the key comes from Doc01's checked `Config`, and `config: Config | None = None` gives two ways in — `None` loads the real settings, a `Config` object is used as-is. `None` is the safe default for "not given" (a default like `load_config()` itself would run once, at import).

Which one should you actually write? Intermediate's single-input chain is all this document's exercises need; the Build Task's Approach 2 reuses Approach 2's optional `config` parameter. `chain.batch(inputs, return_exceptions=True)` is worth knowing exists the moment you're processing more than a handful of inputs through the same chain — a CSV of rows, a list of user messages to classify — where losing every result to one bad row would be a real, avoidable cost, and where requests running concurrently instead of one-by-one genuinely saves time. Not needed for a single-input chain like this one.